Documentation

Table of Contents

1. Baseline System....3
    1.1. AI-Assisted Development....3
    1.2. Initial Architecture....6
        1.2.1. Communication between components....6
        1.2.2. An Overview on the Cryptography mechanisms....6
        1.2.3. The Lack of features....6
        1.2.4. The Sensor....7
            pre_shared_key: Bytes....7
            data_buffer: List....7
            GenerateData() : String....7
            sendData() : Bytes....7
        1.2.5. The Edge Gateway....8
            pre_shared_key : Bytes....8
            p : Integer....8
            g : Integer....8
            dh_private_key : Integer....8
            dh_public_key : Integer....8
            shared_secret : Integer....8
            scloud_aes_key : Bytes....8
            negotiateSession(cloud_service : CloudService) : void....9
            receiveData(legacy_device : LegacyDevice) : Bytes....9
            decryptData(encryptedData : Bytes) : String....9
            reEncryptData(plaintext : String) : Bytes....9
            forwardData(cloud_service : CloudService, reEncryptedData : Bytes) : void....9
        1.2.5. The Cloud Service....10
            p : Integer....10
            g : integer....10
            dh_private_key : Integer....10
            dh_public_key : Integer....10
            shared_secret : Integer....10
            aes_key : Bytes....10
            incoming_buffer : List....11
            negotiate_dh(gateway_public_key : Integer) : Integer....11
            receiveData(encrypted_data : Bytes) : void....11
            processData() : String....11
        1.2.6. Sequence Diagram of the Baseline System....12
    1.3. Baseline Test, Measurements and initial CI/CD Pipeline....13
        1.3.1. Baseline Tests....13
            test_cryptography_unit....13
            test_handshake_math_correctness....14
             test_end_to_end_integration....14
             test_end_to_end_integration....15
                Empty Buffer during date retrial....15
                Null Inputs during Processing....15

Corrupted Data or Invalid Padding....15
1.3.2. Performances of the Baseline System....15
1.3.3. Coverage of the Baseline Tests....16
1.3.4. CI/CD Pipeline....16
2. Testing, Reviewing, Correcting and Improving the Baseline System....17
2.1. Data Corruption....17
2.1.1 Statistical Solution....17
2.1.2. More Integrity Guarantees Required....18
2.1.3. Technical Debt....18
2.2. Message Persistence in Cloud Service (SQLite)....19
2.2.1. Testing....19
2.2.2. Fixing the redundancy....19
2.3. ML-KEM Implementation....20
2.3.1. Testing....20
2.3.2. Performance Comparison....21
2.3.3. Functionalities Comparison....22
2.3.4. Technical Debt....23
2.4. Implementing a “demo”....24
2.4.1. Rich (https://github.com/textualize/rich)....24
2.4.2. Plotext (https://pypi.org/project/plotext/)....24
2.4.3. Testing the TUI....24
2.4.4. Technical Debt....24
The monolithic architecture of *main.py*....25
Lack of thread-safety....25
2.4.4. Resolving The Technical Debt....25
2.4. The CI/CD Pipeline....27
2.4.1. The Monitoring System....27
2.4.2. Building the Deployment Environment....28
2.4.3. Health checks....28
2.4.4. Fixing Docker Volumes....29
3. About the use of AI....30
3.1. AI Usage for the integration of ML-KEM....30
3.2. AI Usage for Demo creation and TUI rejection....31
3.3. AI Usage for building and improving the CI/CD Pipeline....32
4. Remaining Technical Debts....33
4.1. Lack of Authentication....33
4.2. Weak KDF (Key Derivation Function)....33
4.3. Hard-coded parameters....33

1. Baseline System

In this first part we are going to explain how we used the AI to assist us in the development phase of the legacy baseline system. We will focus on relevant prompts, outputs and decisions.

Then we will proceed with the analysis of the baseline system, the initial architecture and features.

1.1. AI-Assisted Development

Instead of generating the entire system with a single prompt, we decided to decompose the system into its three main components:

- The Legacy Device
- The Edge Gateway
- The Cloud Service

And tackle the development of each component with a different prompt, the process has been incremental in order to allow an easy check of the code for each step, the relevant prompts are:

User Prompt:

I was thinking about proceeding by following the UML Class diagram and the Sequence Diagram.

** generateData() can simply be a function that generate a short random sequence of data like this: <String><TimeStamp>.*

** encryptData(string) rely on tru AES*

** sendEncryptedData() → I want to address this function in a simple way, for now, let us make this function put the generated data encrypted bu the previous fanction into a buffer: a list or dynamic array, pick the simplest.*

For now generate this component I will inspect it and then decide how to proceed with the rest

AI Response:

I have generated the `Legacy Device` component exactly as you requested based on your UML and sequence diagrams.

You can review the code here: [`src/legacy_device.py`]

I successfully tested the script, and the encryption works as expected.

Please inspect the component, and let me know how you'd like to proceed with the Edge Gateway!

User Prompt:

Ok now go on with the implementation of the Edge device as described in the Class_Diagram:Phase0.puml

** keep in mind to provide to it the same pre-shared-key of the legacy_device.*

** negotiateSession(): void --> Leave this empty for now.*

** receiveData(encyptedData) : --> must pop encryptedData from the legacy_device buffer*

** decryptData(encryptData) : --> decrypt the received data from the buffer*

** reEncryptData() --> reEncrypt with an Hard-Coded RSA key since for now negotiateSession() is empty*

** forvardData() --> Just print out the re-encrypted data for now.*

For every functionality that is left Empty put a //TODO in the code, when we will create the CloudService then we will figure it out.

AI Response:

I have successfully implemented the Edge Gateway according to your instructions and the sequence from the class diagram. I've also recorded this latest interaction into the `conversation_log.md` inside the `IA Conversation/Phase1/2026-09-11` folder.

You can inspect the code here: [`src/edge_gateway.py`]

User Prompt:

Ok and let us start now with the Cloud Service.

In order to keep it simple, I was thinking of implementing the connection in a way similar with the connection between the gateway and the legacy device. But I have concern about the key_negotiation part.

Moreover since an entire TLS handshake is quite complex since it includes things as CIPHERSUITE and moreover this entire thing will be repleced by the post quantum algorithm, I was thinking about implementing this key agreement as a Diffie-Hellman key exchange (the schoolbook version, that is even simpler than the one really used). Before proceeding with writing the code, tell me what you think about all of this, how can we implement a "fake TCP connection" ?

AI Response:

To simulate a stateful connection (like a handshake) without true network sockets, we can use direct method calls between the objects to act as the "network round-trips". Here is a simple way to do it:

1. The Handshake (Diffie-Hellman Negotiation):

Instead of sending packets, the `EdgeGateway` calls a method on the `CloudService` and passes its DH public key. The `CloudService` returns its own DH public key in response.

** `EdgeGateway` generates its DH private/public keys.*

** `EdgeGateway` calls: `cloud_public_key = cloud_service.negotiate_dh(gateway_public_key)`*

** Inside that method, the `CloudService` generates its own keys, computes the shared secret, and returns its public key.*

** The `EdgeGateway` receives the response and computes the identical shared secret.*

(Boom! Handshake complete, shared secret established.)

2. The Data Transfer:

Once the session is negotiated, we can simulate sending data exactly like we did with the legacy device.

** We can give the `CloudService` an `incoming_buffer` (a list).*

** The `EdgeGateway`'s `forwardData()` method will take the RSA-encrypted (or AES-encrypted using the new DH shared secret) data and append it to `cloud_service.incoming_buffer`.*

** The `CloudService` then pops the data from the buffer and decrypts it.*

By passing the object references to each other (e.g., `gateway.connect(cloud_service)`), they can communicate seamlessly.

Does this architecture sound good to you? If so, I can go ahead and write the implementation plan and the code for the `CloudService` and update the `EdgeGateway` to complete the chain!

As can be seen we stuck with the hint given by the track to keep the all system simple. Then no true TCP connection has been implemented in order to reduce the layers of complexity and the DH key exchange has been implemented as a simple version: no elliptic curves, shared secret computed in a trivial way. The system has been then designed to be functional and easy understandable.

We followed a step-by-step approach in order to have small piece of code to review each time. In this way we could see that everything was correct and understandable. We avoided keeping code that we were not able to understand to avoid as much as possible future technical debt.

The class diagram that the AI followed in order to implement the system will be discussed an analyzed in next section.

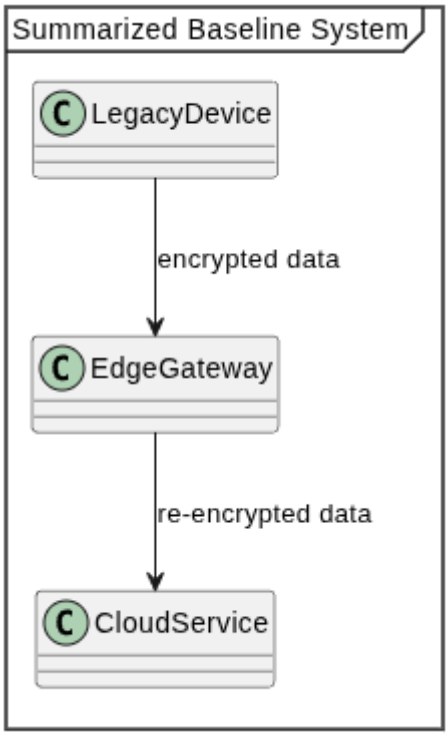
Moreover we need to point out a final thing. We changed the type of string returned by the function *generateData()*, Instead of the random string in the form *<string>|<timestamp>* we decided to go for a meaningful data type, then the string that the function returns, in the end is just a temperature in the form *<temperature>|<timestamp>*.

1.2. Initial Architecture

The Legacy system consist of three main components:

- The Sensor (Represented as “Legacy Device”).
- The Edge Gateway.
- The Cloud Service

1.2.1. Communication between components



In order to keep the system as simple as possible the communication between the devices does not happen via a real TCP connection. In this initial architecture the communication between the devices is implemented via buffers: a sender (e.g. the Edge Gateway) when need to send data to receiver (e.g. the Cloud Service) just put the data into a buffer that follows a FIFO policy (usually implemented as a List object), the receiver is then responsible for retrieving the data from that buffer.

1.2.2. An Overview on the Cryptography mechanisms

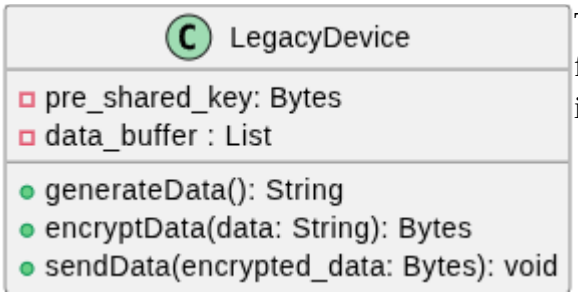
In order to simulate the strict hardware and power constraints typical of legacy IoT sensors, the Sensor have a hard-coded AES symmetric key shared with the Edge Gateway. The Sensor encrypts its data with this static key and transmits it to the Gateway. The Edge Gateway is then responsible for decrypting it and enciphering it again with another key that is established together with the Cloud service thanks to a well-known key-agreement protocol: The Diffie-Hellman protocol. Since this protocol is vulnerable to quantum attacks, ML-KEM will be integrated in future phases of the project.

1.2.3. The Lack of features

Since the first part of the project focuses on modernizing the key agreement mechanism between the Cloud service and the Edge Gateway, extra functionalities such as databases and GUI to visualize the data stored in the Cloud have not been implemented intentionally.

With this system, that aims to mimic a wireless connection, and lacks some advanced functionality, we are both able to start with a simple architecture to understand and leave room for future improvements.

1.2.4. The Sensor



The sensor is the component of the system responsible for capturing data from the environment and sending it to the Edge Gateway.

Here we present a breakdown of its attributes and methods:

pre_shared_key: Bytes

This data is a hard-coded AES cryptography key that the sensor uses to cipher the data before sending it to the edge gateway. These attributes resemble how legacy IOT device works: not enough computing power to do key-exchange, and then a symmetric key is hard coded into them.

data_buffer: List

A data structure that aims to represent a small buffer in which the Sensors hold recorded data before sending them to the edge gateway.

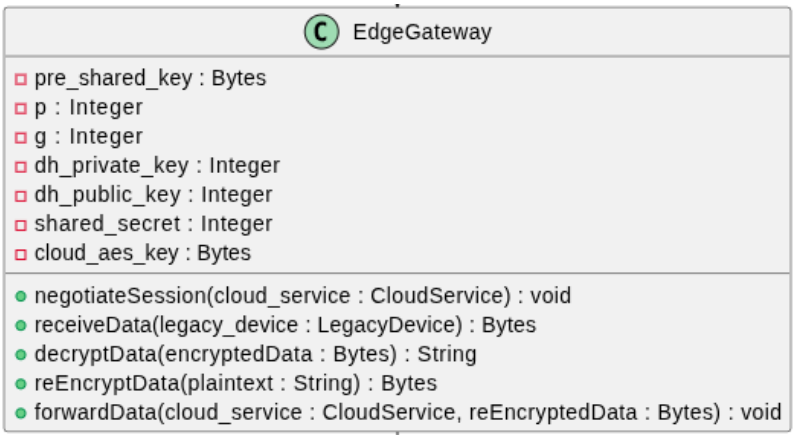
GenerateData() : String

A method to mimic the functionality of the sensor to record data from its surroundings. In the current implementation, this method generates a structured string representing a temperature reading, formatted as `<temperature>|<timestamp>`. This method also includes a probabilistic mechanism to simulate hardware malfunctions by occasionally generating physically impossible temperatures (e.g., erratic readings between -999.0 and +999.0).

sendData() : Bytes

This method just put the encrypted data into the data_buffer data structure in order to simulate the “sending” to the edge gateway.

1.2.5. The Edge Gateway



The Edge Gateway is the core of the system. It is the one in charge of the communication between the Sensor and the Cloud.

Here we present a breakdown of its attributes and methods:

pre_shared_key : Bytes

The exact same hard-coded AES cryptography key that the Sensor has, the gateway uses it to decrypt the data received from the Sensor.

p : Integer

The public modulo that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

g : Integer

The public base that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

dh_private_key : Integer

The ephemeral private key chosen by the Edge Gateway each time a communication session is initiated.

dh_public_key : Integer

It is the result of $(g)^{dh_private_key}$ (mod p) that is sent to the Cloud Service during the key agreement procedure.

shared_secret : Integer

It is the result of modular exponentiation, raising the received public key to the power of its own private key, modulo p.

scloud_aes_key : Bytes

The symmetric key shared between the Edge Gateway and the Cloud Service, used to cipher the messages directed to the Cloud, it is derived via a single application of the algorithm SHA256 to the shared secret.

negotiateSession(cloud_service : CloudService) : void

Initiate and complete the negotiation session of the shared secret with the Cloud Service.

receiveData(legacy_device : LegacyDevice) : Bytes

Method used to simulate receiving data from the Legacy Sensor. It consist into a simple fetch of data from the Sensor Buffer.

decryptData(encryptedData : Bytes) : String

Method used to decrypt the data received from the sensor with the hard-coded AES symmetric key.

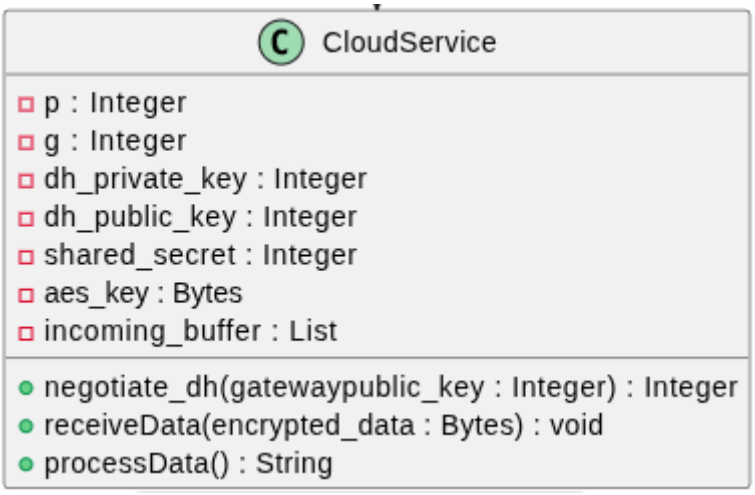
reEncryptData(plaintext : String) : Bytes

Method used to encrypt data with the shared secret established with the Cloud Service. The algorithm used here is also AES.

forwardData(cloud_service : CloudService, reEncryptedData : Bytes) : void

Analogous to LegacyDevice.sendData(), but for sending data to the Cloud Service.

1.2.5. The Cloud Service



The Cloud Service represents the backend infrastructure. It acts as the termination point for the communication with the Edge Gateway and is responsible for final data processing.

Here we present a breakdown of its attributes and methods:

p : Integer

The public modulo that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

g : integer

The public base that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

dh_private_key : Integer

The ephemeral private key chosen by the Cloud Service each time a communication session is initiated.

dh_public_key : Integer

It is the result of $(g)^{dh_private_key}$ (mod p) that is sent to the Edge Gateway during the key agreement procedure.

shared_secret : Integer

It is the result of modular exponentiation, raising the received public key to the power of its own private key, modulo p.

aes_key : Bytes

The symmetric key shared between the Edge Gateway and the Cloud Service, used to cipher the messages direct to the Cloud, it is derived via a single application of the algorithm SHA256 to the shared secret.

incoming_buffer : List

A FIFO data structure simulating a receiving network socket where the Edge Gateway deposits encrypted payloads.

negotiate_dh(gateway_public_key : Integer) : Integer

This method is called by the gateway to complete the handshake. It generates the cloud's ephemeral keys, computes the shared AES secret using the gateway's public key, and returns the cloud's public key to the gateway.

receiveData(encrypted_data : Bytes) : void

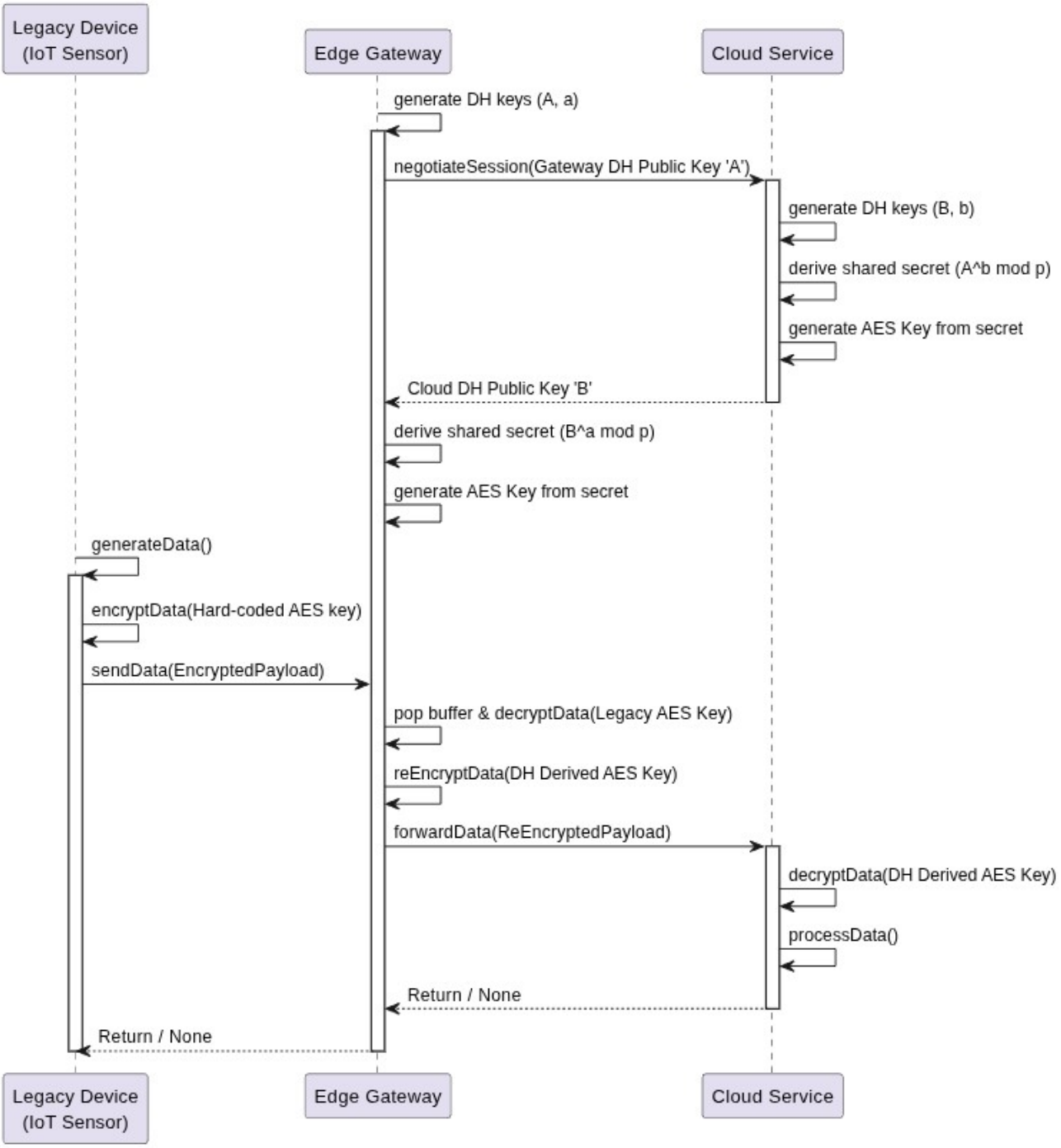
Method used to simulate the reception of data from the Edge Gateway.

processData() : String

Method used to simulate the processing of data, used to simply decrypt the received data from the Edge Gateway.

1.2.6. Sequence Diagram of the Baseline System

The following diagram shows how the system behaves when a data entry is generated from the sensor. This simple communication involves a single message that travels from the Sensor to the Cloud Service.



1.3. Baseline Test, Measurements and initial CI/CD Pipeline

Before introducing the post quantum cryptography key exchange and any other feature in the system, it is necessary to ensure that the legacy system functions correctly. Moreover it is necessary to provide some metrics in order to compare the impact of the future ML-KEM integration with respect to the current key-exchange mechanism.

Then we will proceed with the following test strategy concerning the baseline architecture.

This section aims to document the initial test suites and the first implementation of our CI pipeline.

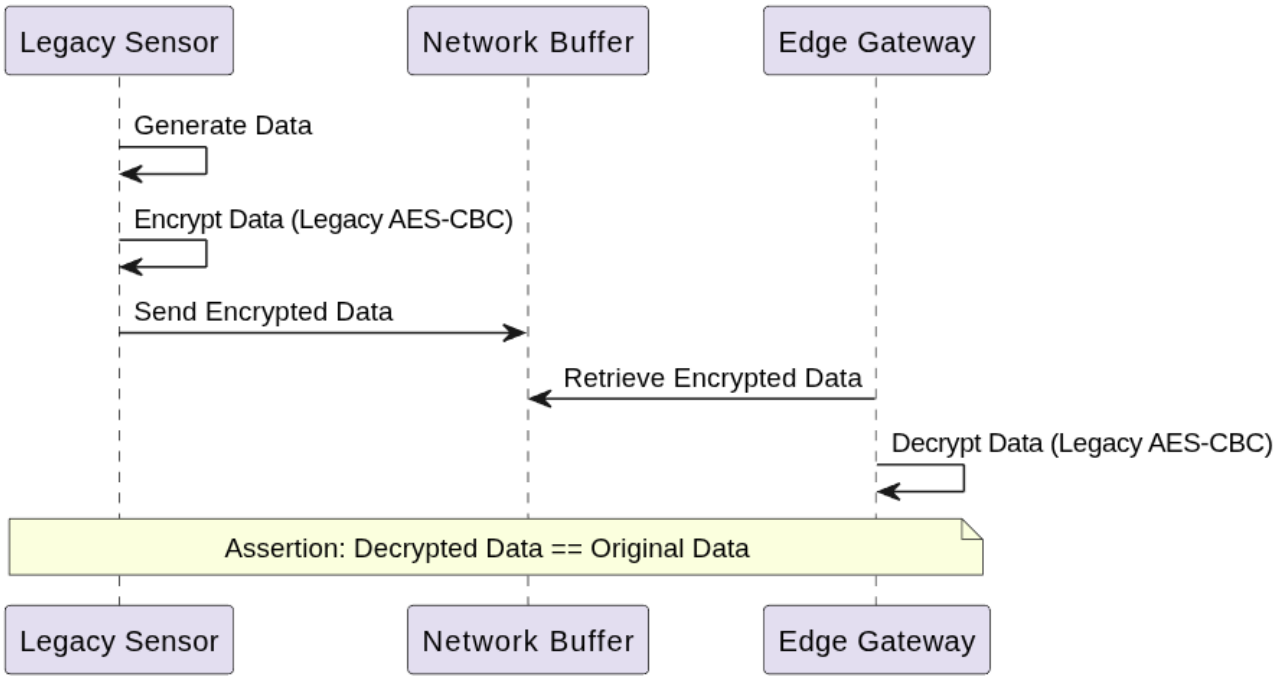
1.3.1. Baseline Tests

The baseline test suite is contained in *tests/test_baseline.py* and it aims to validate the core functionalities, cryptographic correctness and end-to-end integration of the system components. Moreover it sets also a performance baseline for the system that will be compared with analogous metrics at the end of the work.

Now we proceed to show and explain the test cases present in our suite.

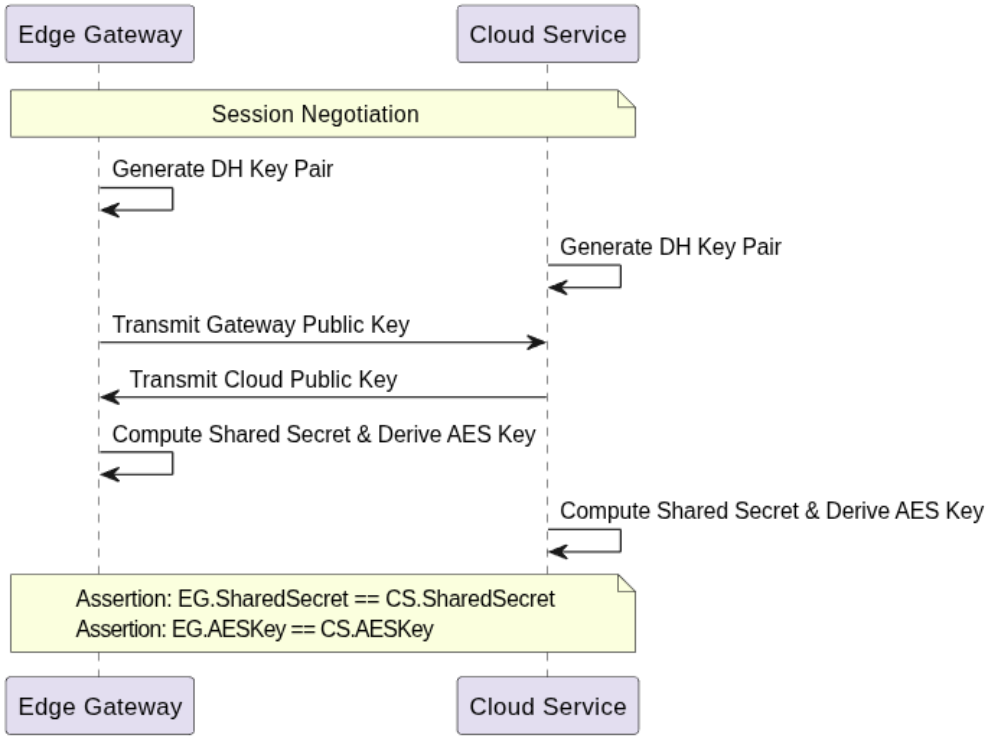
test_cryptography_unit

The purpose of this test is to verify tat the data encrypted by the Legacy Sensor using its hard-coded AES key can be successfully and deterministically decrypted by the Edge Gateway. The test works as follow:



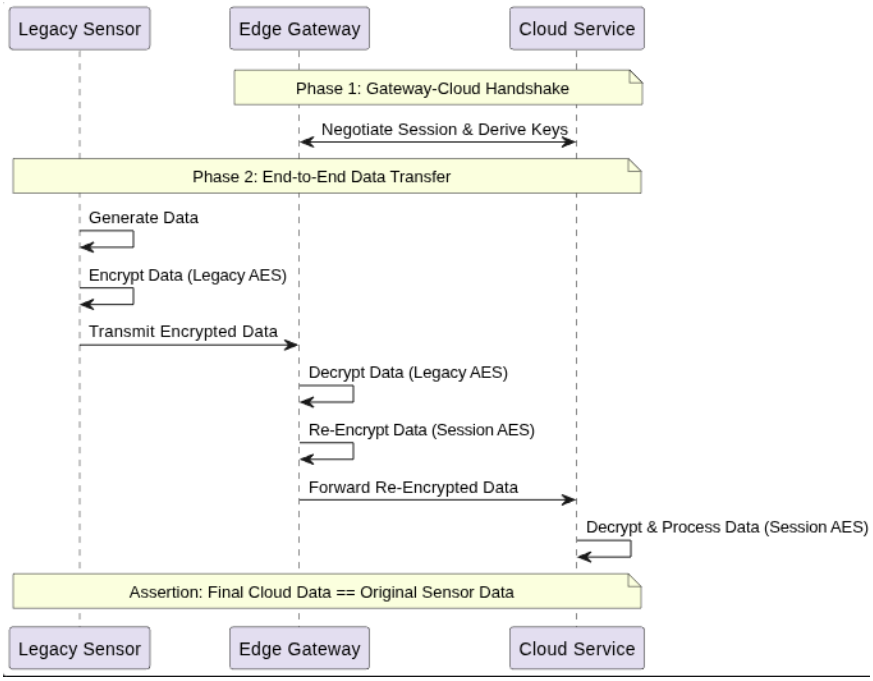
test_handshake_math_correctness

The purpose of this test is to verify the mathematical correctness of the Diffie-Hellman handshake between the Edge Gateway and the Cloud Service. The test works as follow:



test_end_to_end_integration

The purpose of this test is to verify an entire operational flow to ensure that the data correctly goes from one end of the system to the other. Moreover this test is also used to extract metrics about the system. The test work as follow:



test_end_to_end_integration

The purpose of this test is to verify the behavior of the system when edge cases occur and how eventual errors are handled. The exact edge cases validated by the test are the followings:

Empty Buffer during date retrial

If the Edge Gateway ties to fetch data from the receiving buffer when there are no data, it is expected that the function receiveData() return Node.

Similarly if the Cloud Service attempt to retrieve a data from its buffer when it is empty it ie expected that the function returns None.

Null Inputs during Processing

If null inputs are passed to the decryption and re-encryption primitive of the Edge Gateway it is expected that the component is capable of handling it and returns None for this type of invalid inputs.

Corrupted Data or Invalid Padding

In this part of the test we make the Device generate 32 Byte of garbage data, simulating a scenario where data is corrupted, and verify that the Cloud Service is actually capable of catching the failure and return None as expected when trying to decipher it.

1.3.2. Performances of the Baseline System

To critically evaluate the improvement in terms of metrics derived by the introduction of ML-KEM we will first of all measure the performances in terms of memory and time required by the Diffie-Hellman protocol. The metrics we plan to keep track of, in order to compare the two implementations, are:

- Handshake Latency (ms)
- End-to-End Latency (ms)
- Public Key Size (Bytes)
- Ciphertext Size (Bytes)

The test suite documented above has been designed in order to produce metrics relative to the system and the coverage, they can also be retrieved as outputs of the CI pipeline that we will described further on.

In the following table we show the metrics recorded for what concern the baseline system:

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher text Size (Bytes)	48
Re-encrypted Cipher text Size (Bytes)	48

1.3.3. Coverage of the Baseline Tests

Module	Executable Statements	Missed statements	Coverage
src/cloud_service.py	39	0	100%
src/edge_gateway.py	54	0	100%
src/legacy_device.py	28	0	100%
TOTAL	121	0	100%

1.3.4. CI/CD Pipeline

The CI pipeline leverages GitHub Actions to automatically run tests and report coverage. In this initial implementation the triggers are:

- push on the main branch.
- Pull requests targeting the main branch.
- Workflow dispatch that allows the pipeline to be triggered manually from the GitHub web interface in order to test the configuration.

The pipeline runs the test suite and captures coverage and performance metrics. A mock CD phase has been put in place, in future work a proper one will be implemented.

2. Testing, Reviewing, Correcting and Improving the Baseline System.

In this part we will report the process of testing, reviewing, correcting and improving the baseline system.

This part will involve also the integration of ML-KEM into the system.

2.1. Data Corruption

We noticed that the baseline system does not perform any kind of data integrity validation through the entire pipeline from the legacy sensor to the cloud service. This mean that the Cloud could end up receiving corrupted data (think about a sensor malfunction, a transmission problem or someone intentionally tampering with the transmitted data).

Then in order to end up filling the Cloud with unreliable data we need to solve the issue. The adopted strategy has been the following:

- Expand the functionalities of the Edge Gateway in order to drop the “bad packets”.
- Expand the functionalities of the Edge Gateway such that it is able to keep a log of the discarded packets.
- Expand the functionalities of the Cloud Service such that it can rise an alarm of some kind if it do not receive update from the sensor for a long time.

After implementing these changes we updated the test suite accordingly. test_baseline.py has been updated in only to test only the “happy paths”, i.e. all the cases in which the packets are properly composed. It has been created a new test suite: test_failure_handling.py that instead only verifies the handling of corrupted packets.

2.1.1 Statistical Solution

While running the test suite we discovered that some malformed packets are still able to slip through. This happen because the legacy system relies on AES-CBC that does not provide any integrity guarantees. Since the Sensor is a legacy system and we cannot apply fixes here, we must surrender to the idea that fixing this issue is impossible. However we can mitigate it as follow:

First of all we can make the validation performed by the gateway more strict in this way: instead checking only that the temperature is a float as the current implementation does we can enforce a regex. Another measure to reduce the probability that malformed records goes throw is to verify that the timestamp falls in a certain interval of time. These measures won't catch corruption where 17.5flips to 16.5 but will drastically reduce the number of undetected corruptions. Then since have a 100% success rate is not feasible we must set a threshold for our tests. We say that we are satisfied (our test passes) if the system catches 80% of the malformed packets.

While conducting further tests we also notice that the probability of a message of passing through depends also on the admitted time interval of the Edge Gateway, tighter is the interval, more probable is to spot corrupted package, if the interval is too tight good packets may be discarded.

Finally to ensure the threshold is respected we made the test suite run about 100 times to ensure as much statistical coverage as possible.

2.1.2. More Integrity Guarantees Required

Since now we have talked only about the communication between the Sensor and the Gateway, now it is time to take into account also the path between the Gateway and the Cloud Service.

As first implementation to enforce integrity guarantees between the Gateway and the Cloud we used CR32 checksum. The problem is that this approach ensure integrity only against accidental data corruption (e.g. network errors and interferences) and then it is not a good method to enforce integrity when for what concern intentional data tampering.

For this reason the AES-CBC mechanism previously used has been substituted with an authenticated mode of encryption: AES-GCM in order to ensure both confidentiality and integrity.

2.1.3. Technical Debt

Since the Communication between The sensor and the Gateway still relies on AES-CBC in this part of the system the information may be subject to tampering or networks error, this represent an integirity problem, that cannot be solved unless the Sensor itself is replaced.

Details can be seen in:

- *patches/AES_GCM_between_edge_gateway_and_cloud.patch*
- *patches/data_corruption_handling_sensor_to_Gateway.patch*

2.2. Message Persistence in Cloud Service (SQLite)

The Cloud Service in the baseline system do not act like a real cloud, in the sense that currently it lacks the core feature of a cloud service that is the persistent memorization of the information.

Sensor telemetry must be reliably stored in a database for historical queries, analytics, and audits.

To solve this issue, the adopted strategy has been the following:

- Introduce persistent storage into the CloudService component using Python's built-in sqlite3 library (requiring zero external dependencies to keep the architecture simple).
- Automatically generate a database table (messages) upon service initialization to store the temperature, timestamp, and original raw message.
- Ensure that the save_message() function is only triggered *after* successful AES-GCM decryption and strict semantic validation, guaranteeing that only healthy data is ever stored.

2.2.1. Testing

After implementing the persistent storage, we expanded our testing coverage. A dedicated automated test (test_cloud_database_persistence) was added to verify the correct insertion of records in the database. This test verifies the correct creation of the table schema and data types. More importantly, it asserts that the Cloud Service completely rejects malformed payloads and refuses to execute the SQL insertion, proving that our database is entirely immune to poisoning from transit corruption or sensor malfunctions. It also verified the end-to-end integration: simulating a legacy sensor reading that successfully traverses the Edge Gateway, gets verified by the Cloud, and finally rests securely in the SQLite database.

2.2.2. Fixing the redundancy

During a subsequent static code review, we identified significant operational redundancy within the SQLite integration. Specifically, the data parsing and schema generation logic were being duplicated on every transaction. We refactored this segment to streamline the data pipeline, eliminating the redundant operations and keeping the entire code base cleaner.

Details can be seen in:

- *patches/sqlite_message_persistence_cloud_service.patch*
- *patches/db_redundancy.patch*

2.3. ML-KEM Implementation

We integrate PQC measure by adopting an hybrid scheme with the DH key exchange already in place. In this way we followed the best-practice standard for the implementation of modern PQC.

The ML-KEM implementation implied several changes and addition to out baseline system that we are going to document here:

First of all we needed to find a library that implements ML-KM, this approach is the best in terms of security since try to write a cryptographic library from scratch is extremely error-prone and would expose the system to severe vulnerabilities, the we relied on the *mlkem Python package* that provided us the official FIPS 203 ML-KEM specification.

As second step we modernized the Cloud Service (*cloudservice.py*) to handle the new hybrid protocol. We had to change the signature of the legacy method *negotiate_dh()* to *negotiate_session()* in order to avoid ambiguities and accommodate the new parameters (the ML-KEM encapsulation key). The method now:

- Compute the DH shared secret.
- Performs ML-KEM encapsulation to generate the ML-KEM shared secret an chipertext.
- Derives the final AES-GCM key by concatenating the DH shared secret and the ML-KEM shared secret via SHA-256.

The third step of this phase was the modernization of the Ege Gateway (*edge_gateway.py*). We modified the method *negotiateSession* to initiate the hybrid protocol that now:

- Generates the ML-KEM encapsulation/decapsulation keys.
- Generates the DH keys.
- Send the keys to the Cloud Service.
- Performs decapsulation upon receiving the ML-KEM cipher text.
- Mixes the retrived ML-KEM secret with the classic DH secret to derive the identical AES-GCM key.

2.3.1. Testing

The previous tests passed flawlessly, then the regression test was successful, we introduced another test suite (*test_mlkem.py*) just to test the basic functionality of the adopted library (it does not invoke any method of the system).

We needed to update the baseline test suite. It was considering only the DH key exchange in the measuring of the metrics, from now on it consider also the metrics of ML-KEM.

2.3.2. Performance Comparison

These are the metrics from the baseline System

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher text Size (Bytes)	48
Re-encrypted Cipher text Size (Bytes)	48

These are the same metrics with the introduction of the Hybrid Scheme (ML-KEM + DH):

Metric	Value
Handshake Latency (ms)	75.8205
End-to-End Latency (ms)	8.1892
Gateway DH Puclic Key Size (Bytes)	256
Gateway ML-KEM EK Size (Bytes)	1184
Gateway Total Payload Size (Bytes)	1440
Cloud DH PK Size (Bytes)	256
Cloud ML-KEM CT Size (Bytes)	1088
Cloud Total Payload Size (Bytes)	1344
Ciphertext Size (Bytes)	32
Re-Encrypted Ciphertext Size (Bytes)	43

→ EK stands for (Encapsulation key), it is the conceptual “Public key” of the ML-KEM key agreement. It is the key sent to Cloud.

→ CT The encrypted packet that the Cloud send to the Gateway once it received the EK.

→ The Gateway’s payload consists of both its DH Public Key + the ML-KEM EK. The Cloud’s payload consists of its DH Public Key + the ML-KEM CT.

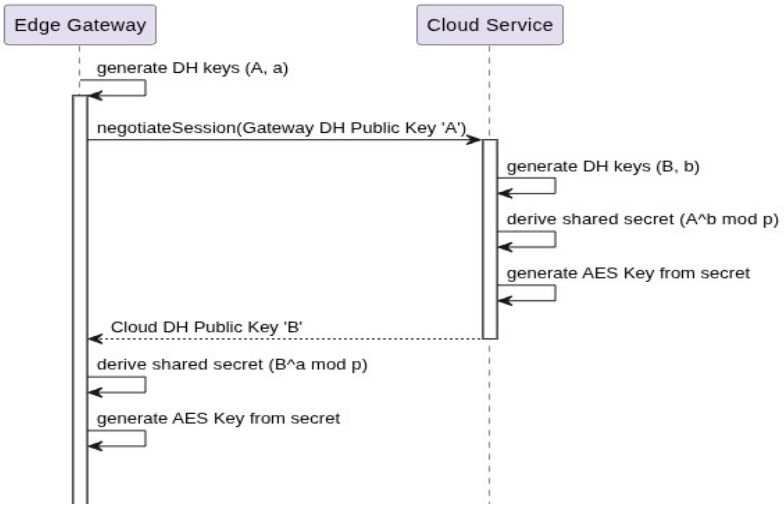
→ Ciphertext Size is the size of the packets that travel between the sensor and the Edge Gateway.

→ Re-Encrypted Ciphertext is the size of packets that travel between the Edge Gateway and the Cloud Service.

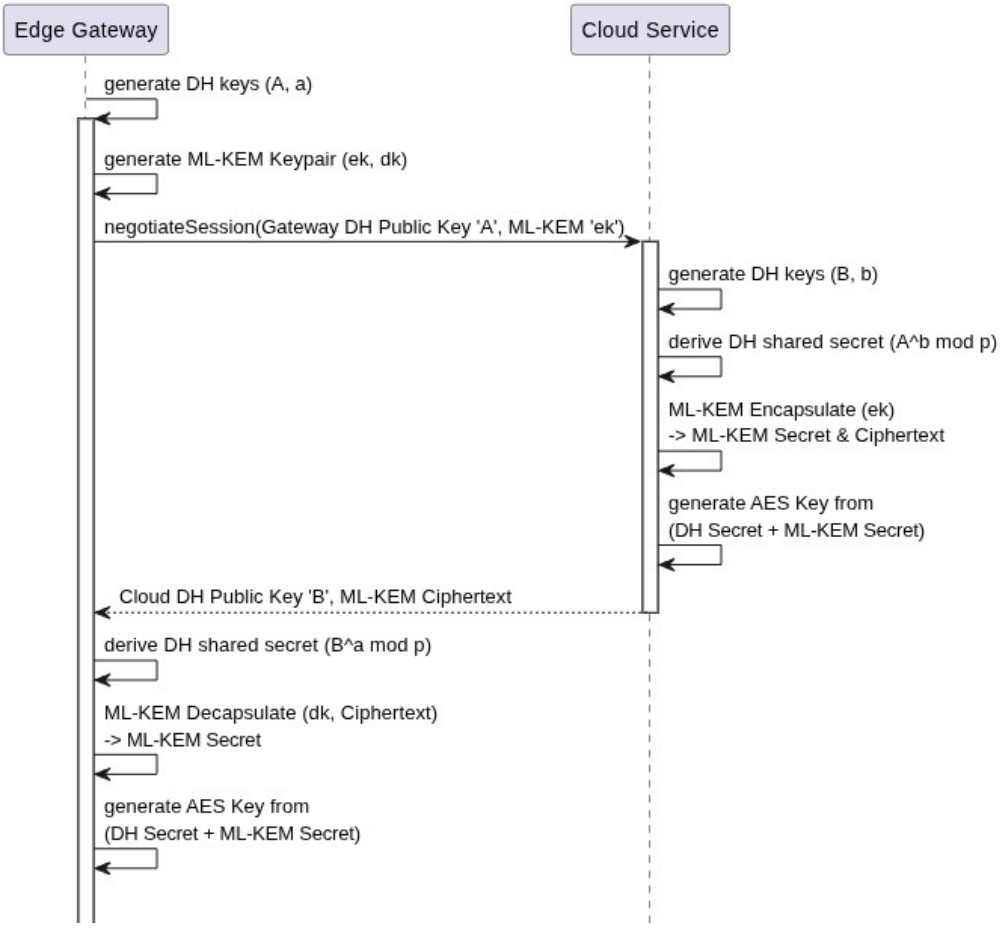
2.3.3. Functionalities Comparison

Here we just see a comparison on “how the system works” before and after the introduction of the Hybrid scheme:

Before: same diagram of the document *Baseline analysis – Initial architecture*



After: Corresponding execution flow after the introduction of the Hybrid scheme:



Details can be seen in:

- *patches/ML_KEM_integration.patch*

2.3.4. Technical Debt

The entire key exchange remain non-authenticated, the system lacks any form of Public Key Infrastructure and digital signatures, then it is vulnerable to Man-In-The-Middle attacks in both directions: Gateway to Cloud and Clod to Gateway. Then an attacker that gest the public parameters is able to impersonate both ends of the communication.

2.4. Implementing a “demo”

In order to showcase the functionality of the system in real-time, we implemented a multi-threaded architecture with a live dashboard.

The demo utilizes three background threads running concurrently.

- The first thread runs the legacy Sensor, which continuously generates and buffers telemetry data.
- The second thread runs the Edge Gateway, which is responsible for fetching the data from the Sensor's buffer and securely forwarding it.
- The third thread runs the Cloud Service, which decrypts the incoming payloads and updates global statistical metrics.

Finally, the main thread manages a TUI (Terminal User Interface). The TUI visualizes the pipeline's operation by displaying a live temperature graph, a log of incoming data, computed statistics, and a visual alarm that triggers if the sensor drops offline.

In order to construct the TUI we added the following new dependencies to the code:

2.4.1. Rich (**https://github.com/textualize/rich**)

It is used to manage the layouts, panels and the updating of the terminal screen. It is a library used to format the terminal, makes easy to add color, bars and tables.

2.4.2. Plotext (https://pypi.org/project/plotext/)

It is used to generate the graph. It is a library that allows for scatter, line, bar, histograms and date-time plots.

2.4.3. Testing the TUI

While letting run the GUI we noticed that, when simulating on purpose a system failure (setting corruption_probability or failure_probability to 1) the TUI stayed simply quiet without reporting anything. This is not the intended behavior because while developing previous part of the system (the Cloud Service) we designed the system to signal if something was broken.

The problem was due to the fact that the thread running the cloud was not considering the function *sensor_health()* to check the state of the connection.

2.4.4. Technical Debt

We have identified several possible issue in introducing this demo:

The monolithic architecture of main.py

Since all of this has the only purpose to showcase the functionality of the system we overlooked the separation of concern among different components. This resulted in having the *main.py* artifacts that does a lot of stuff: it has logic necessary to the key negotiation, logic to manage the thread and moreover it contains logic related to the TUI.

This is a well-known anti-pattern. Already in this state the artifact is difficult to read and maintain, if more functionalities are added to the TUI without addressing this problem first the codebase will end-up having an unreadable bottleneck.

It is a best practice that the UI logic is keept separated from the rest of the code.

Lack of thread-safety

The components of the system communicate with the TUI via global variables and there is no lock mechanism on these data structures. Then multiple threads have concurrent access to the same data structure. Since our system is pretty simple we do experience race conditions but in a more complex system this could result in a problem. For instance, if the same TUI needs to take into account data from different sensors the chronological order of the data may not be respected at all. To solve this a lock mechanism should be introduced.

This problem in theory is also present for what concern the exchange of packets between the three components of the system, but in this case since the system is properly synchronized via hard-coded time intervals (another very bad practice), the problem is no present.

Details can be seen in:

- patches/TUI_Implementation.patch

2.4.4. Resolving The Technical Debt

While proceeding with the works, during the enhancing of the CI/CD pipeline (see the next section of this document for details). We made the decision to completely remove the TUI from the codebase . While it was useful as brief demo for visualizing the execution flow of the system, it introduces too much complexity to be efficiently handled and maintained, moreover, most of the job of the TUI is done by the monitoring system we implemented (see the next section of this documents for details). The removed the TUI, and consequentially solved great part of the technical debt in the following way:

- We removed the rich and plotext dependencies, drastically reducing the application's footprint and simplifying the Docker container.
- We stripped out the global data buffers, layout logic, and the UI rendering loop, neutralizing the first thread-safety risks analyzed in the section: “T*he monolithic architecture of main.py”*
- We introduced Python's standard logging module to allow the application to run silently in the background, outputting structured logs to standard output.

Moreover in this way we also end-up with cleaner and readable code.

Details can be seen in:

- patches/removing_the_TUI.patch

2.4. The CI/CD Pipeline

The starting point for enhancing the CI/CD pipeline is the baseline system where we had a mock pipeline that just execute the tests and has a placeholder for the docker orchestration we aim to generate. While in the previously we already set up an initial docker environment to run the demo of the system here we are going to add the live monitoring system, fix eventual issue with the deployment and the docker orchestration.

2.4.1. The Monitoring System

For the monitoring system we relied on Prometheus, an open source monitoring and alerting toolkit.

After importing the library in our *main.py* we defined three specific metrics:

sdmo_packets_total

This is a counter that racks the number of packets that have reached the Cloud Service. In Prometheus this data type can only go up and it is useful for an eventual operation team to measure the throughput of the system.

sdmo_temperature_celsius

This metric tacks the live temperature reading currently being processed by the system, the type of data used here is the Gauge. Differently from the counter this kind of data can go up and down.

sdmo_errors_total

This is another counter that acts as an health indicator, It increments every time the Cloud Service detects a failure (i.e. it does not receive packets for long time). This metric is useful because it can be used to launch automated alerts.

These metrics are exposed on an HTTP server that is opened on the port 8000 running in the background of the main thread. Then the metrics can now be seen on *localhost:8000/metrics*. The output is not intended for humans, we are going to address this problem in the next section.

Details can be seen in:

- *patches/Prometheus_initial_implementation.patch*

2.4.2. Building the Deployment Environment

In order to build the deployment environment we are relying on Docker Compose. We will simply add one more container to our current deployment architecture. Up to now we have worked only with a single container that runs the application. In order to integrate properly the monitoring system we will spin up another container that is responsible for running the the Prometheus monitoring server. The two containers will then talk to each other and Prometheus will be then able to show the metrics in an human-readable format useful for actively monitoring the system.

So, first of all, we create a configuration file for Prometheus (*prometheus.yml*) that specify to the Prometheus instance where to look for metrics (the “sdmo app” on the port 8000).

Secondly we create the docker compose file (*docker-compose.yml*) that is the core of our deployment that defines the two services.

With this system now we should have a working system, with a monitoring-system that is actually something human-readable, indeed the Prometheus dashboard for the monitoring can be seen at *localhost:9090*.

Details can be seen in:

- patches/Deployment_Environment.patch

2.4.3. Health checks

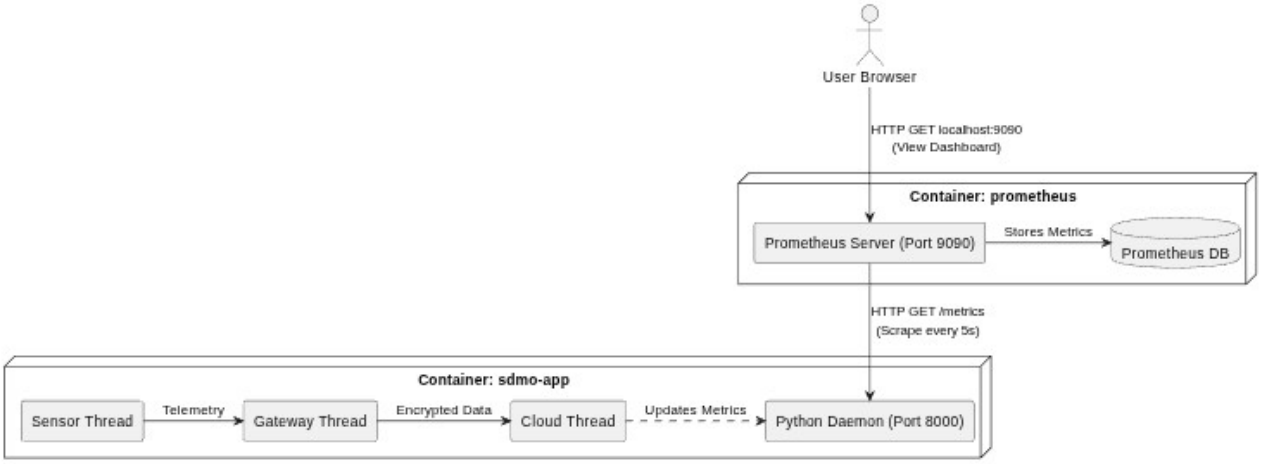
Now comes the moment to improve our CI/CD pipeline with some health checks. Currently the pipeline (*ci.yml*) tries to execute the tests and just have a placeholder for the docker orchestration we implemented in the previous step. However just because our containers are running this does not mean that the application inside it is working. Indeed the tests could pass and the docker orchestration may also build flawlessly but what happens if the whole system enter some sort of infinite loop that we did not spot before or remains blocked for other reasons? To solve this we will rely on a “health check” previously putted in out docker compose configuration that the CI/CD pipeline will use to know the status of the application.

It works as follows:

Every ten seconds the application tries to fetch the metrics from the Prometheus metrics page, if the Python application is running correctly the Prometheus client will respond with an HTTP 200 OK message. When this message is sent then the application is considered “healthy”. This message exchange is tried three times before the application is considered “unhealthy”, i.e. broken.

Then, with this extra check, the GitHub action will fail not only when the test suite will fail, but also when there are problems in the deployment, effectively protecting the main branch.

The following diagram, show the interaction between the two containers, and how the browser visualize the data from the monitoring system.



Details can be seen in:

- patches/Deployment_Environment.patch

2.4.4. Fixing Docker Volumes

Currently, the SQLite database and Gateway logs generated by the system are stored inside the ephemeral Docker container. Then all the data produced by the application are deleted when the container dies.

To make this a true production-ready architecture, we need to map those directories out to your host machine using Docker Volumes.

In this way the logs of the Edge Gateway will be returned, and in case of eventual transmission failures, these can be used to address eventual problems and the SQLite database build by the Cloud Service is stored on the machine that execute the application, in this way further analysis on the produced data are possible.

Details can be seen in:

- *patches/Docker_Volumes.patch*

3. About the use of AI

In this section we are going to dive deeper in the use of AI for what concern the most important activities documented in section 2 (Integration of ML-KEM, implementation of the “demo” and the building and enhancing of the CI/CD pipeline).

3.1. AI Usage for the integration of ML-KEM

The AI was provided with the following key constraints: preserve sensor immutability, enforce pure-Python dependencies, maintain test suite compatibility and adhere to hybrid migration standard combining classical DH with ML-KEM.

The primary prompt was:

“We are in Phase 2, I am doing the system modernization. The system is ready to implement ML-KEM. The final scheme must be a Hybrid scheme that combines DH and ML-KEM following the best-practice of mixing the two secrets into a single one when PQC is introduced. Update the Edge Gateway and the Cloud Infrastructure in order to integrate ML-KEM without breaking regression tests.”

Follow-up minor prompts requested to maintain the signature of the methods used to establish the key exchange and to update the baseline test suite in order to split DH and ML-KEM metrics into separate rows.

The AI selected the pure-Python *mlkem* package implementing then NIST FIPS 203 (ML-KEM768). Updated the Cloud Service and the Edge Gateway to exchange ephemeral 2048 bit DH keys and ML-KEM encapsulation keys. It also propose upgrading transport encryption to AES-256-GCM. Following developer feedback, it updated the baseline test suite to record the sizes of the exchanged key and ciphertext in the metrics recorded by the tests.

The developer in this phase rejected the AI proposition to update the sensor code to introduce TLS 1.3 due to the fact that the modernization operations should not involve the legacy sensor, since it is the legacy component of the system it can’t be touched.

3.2. AI Usage for Demo creation and TUI rejection

The original plan was to demonstrate the functioning of the pipeline thanks to visual monitoring.

The main prompts used to address this matter have been the followings:

"*I would like to enhance main.py in order to showcase the system pipeline […] A thread that runs legacy_device.py, another that runs edge_gateway.py, and cloud_service.py running in another thread […] I was thinking about a facy TUI separated into two parts, the top showing a graph of teperatures, the bottom divided into live incoming data and statistics (average, maximum, minimum)*"

"*I was looking at main.py, would the GUI work properly if I tweak the failure_probability and corruption_probability? The TUI does not show the alarm message*"

This part has being turbolent, the AI restructured the *main.py* artifact using three background threads coordinated by *time.sleep()* in order to render the dashboard. It introduced new dependencies (*rich* and *plotext* libraries).

The multi thread pipeline structure as been accepted by the developer but the following analysis of the code reveled that the AI omitted the checks for corrupted packets, then the developer modified the code to enforce continuous alarm evaluation.

Further analysis brought the developer to reject the monolithic architecture introduced to implement the TUI, this lead to a massive reduction of code complexity and also the dependencies were deleted. The monolithic TUI was replaced using standard logging and Prometheus.

3.3. AI Usage for building and improving the CI/CD Pipeline

The main prompts used to address this matter have been the followings:

“Create a GitHub Actions CI pipeline in.github/workflows/ci.yml that executes pytest with coverage, saves test metrics to tests/voverage.txt, and builds the container image”

“I was browsing the internet to find a way to implement a monitoring system for the project, I found out that a standardized way is to use Prometheus. Could you show me if and how it is possible to integrate such a tool in this project?”

The rest of prompts focused on configuring Docker volume mounts for the outputs and resolving CI failures caused bu deprecated container commands since the developer was using an outdated version of docker.

The AI generated a multi-staged Dockerfile capable of doing health checks of the system. A *docker-compose.yml* file to orchestrate the main app alongside with a phrometheus container was generated.

The developer accepted the majority of the generated artifacts but had to take action for what concerned the docker volumes by adding host bind mount to to guarantee the persistence of the Cloud Database and the gateway logs. The developer also rejected deprecated standalone *docker-compose* binary suggested in early prompts, enforcing modern *docker compose* syntax.

4. Remaining Technical Debts

In this section we are going to document all the technical debts that remain in our system and are not correlated to any of the previous maintenance and operations activities.

4.1. Lack of Authentication

While confidentiality is ensured via ML-KEM768 and AES-256-GCM the system does not implements signatures or certificates, then both the Cloud system and the Edge gateway have not any ways to prove who they are. Then an attacker can just intercept the connection and perform man in the middle attacks.

4.2. Weak KDF (Key Derivation Function)

The cryptographic key bot the Cloud and Gateway rely on is derived by applying SHA256 to the shared secret among the two components of the system. While this yields a perfectly valid AES key, using a standard hash function as a KDF is considered technical debt in modern cryptography because SHA256 lack context binding.

4.3. Hard-coded parameters

Both the Edge Gateway and Cloud Service come with hard-coded parameters for what concern the DH key exchange, this is another bad practice for what concern cryptography. In order to fix this the initial handshake should be fixed in such a way that the two components of the system agree on a public base for the exponentiation. Moreover also the private exponent that is hard-coded in both the components should be generated at random. Fixing this is effective only if authentication mechanism (like PKI) are provided, otherwise the system remains vulnerable to Man In the Middle attacks.